A Study on Applying Large Language Models to Issue Classification

Jueun Heo
Department of AI Convergence Engineering
Gyeongsang National University
Jinju, Republic of Korea
juandeun@gnu.ac.kr

Seonah Lee
Department of AI Convergence Engineering
Department of Software Engineering
Gyeongsang National University
Jinju, Republic of Korea
saleese@gnu.ac.kr

***Abstract*—Prompt-based large language models (LLMs) have demonstrated their ability to perform tasks with minimal or no additional training data. In the context of issue classification, researchers have actively explored the capabilities of LLMs in classifying issue reports. However, existing studies still face limitations in accuracy. This study replicates an LLM-based issue classification study using GPT-3.5 Turbo and explores variants, such as adopting different models like Llama 3.1 8B and GPT-4o. Experimental results show that the classifier fine-tuned with GPT-3.5 Turbo still yields the same accuracy as shown in the original research and that the classifier fine-tuned with Llama 3.1 8B (0.8004) yields an F1-score of 0.0535 lower than that of the classifier fine-tuned with GPT-3.5 Turbo (0.8467). On the other hand, the classifier with GPT-4o (0.8639) yields an average F1-score 0.01 higher than that of the classifier fine-tuned with GPT-3.5 Turbo (0.8467). Additionally, the project-agnostic classifier fine-tuned with GPT-4o yields the highest F1-score of 0.8680. These findings contribute to advancing LLM-based issue classification by providing experimental insights into the accuracy of LLMs in this issue classification task.**

***Index Terms*—Issue, Classification, GPT-3.5, GPT-4, OpenAI, Llama 3.1 8B, Instruction Tuning**

I. Introduction

Issue reports are a communication channel for users to report bugs, questions, and feature requests that they find while using the software. Issue reports also allow software developers to identify problems in their projects, development, and maintenance. Managing and handling issues enables rapid maintenance of software and promotes user-driven development and enhancement. Issue management effectively plays a key role in ensuring that software is a competitive product in the marketplace.

However, in large software projects, many users report a wide variety of issues. It takes a lot of time and effort for managers to classify those issues. To accurately classify these issues, the manager is required to have a deep understanding of the context of the software's development status, environment, technology, schedule, and more, as well. Therefore, researchers have been working on automatically categorizing issue reports with AI techniques.

Issue classification approaches mainly consist of preprocessing issues, converting them into embedding vectors, and then classifying them into categories. Early studies utilized deep learning models such as RNN [1], LSTM [2], and FastText [3] to implement issue classifiers, but these models relied on static embeddings and were limited in their ability to fully capture the contextual meaning of words.

Therefore, recent research in issue classification has centered on neural network-based deep learning models and Large Language Models (LLMs). Especially, Transformer-based models, BERT [4] and GPT [5], utilize self-attention mechanisms to effectively grasp contextual meaning by considering all associations between words. BERT and GPT are large-scale architectures with hundreds of millions to billions of parameters. These transformer-based models perform well in various natural language processing tasks. BERT [4] and its variants such as RoBERTa [6] have been widely used in the issue classification task to understand and categorize issue reports based on sentences. Also, domain-specific pre-trained models CodeBERT [7], BERTOverflow [8], seBERT [9], and IssueBERT [10] have shown the best accuracy in issue classification [10].

However, in recent years, GPT models have been able to perform a variety of natural language processing tasks with just a prompt. To achieve that, GPT models have used large amounts of data, scalable architectures, and various learning techniques such as in-context learning [11], instruction tuning [12], few-shot learning [5], and Reinforcement Learning with Human Feedback (RLHF) [13]. As a result, GPT models can also be used for classification problems to achieve good performance with less domain data.

In this paper, we replicate Aracena et al.'s study [14] that utilized OpenAI's APIs to classify issue reports across five GitHub projects on GPT-3.5 Turbo. We extend their work by collecting additional data of six projects, experimenting with different LLMs such as Llama 3.1 8B and GPT-4o, and building an project-agnostic model. Key contributions of this paper are:

This research was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF) grant funded by the Ministry of Education (NRF-2021R1A2C1094167).

2643-7171/25/$31.00 ©2025 IEEE
DOI 10.1109/ICPC66645.2025.00022

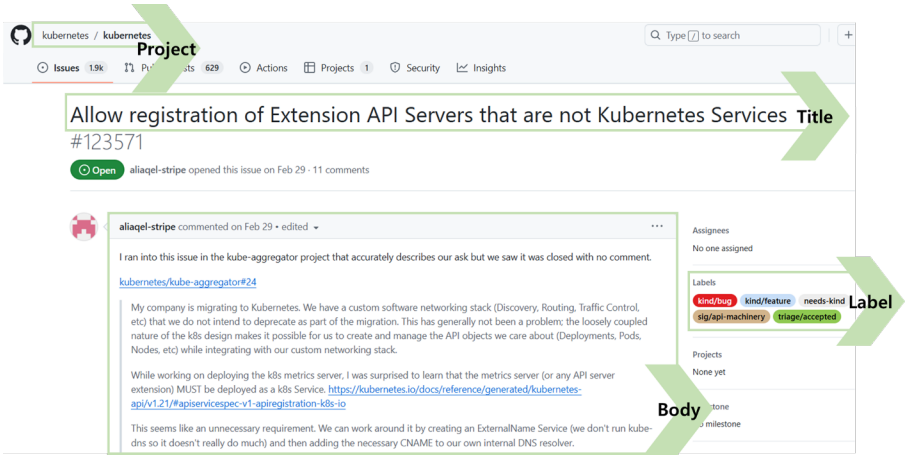

Fig. 1. An Example of Issue Report

- **Collecting data from additional six open source projects**: We collected data from six additional open source software projects on GitHub to expand the dataset for issue classification tasks.
- **Comparing the accuracy of GPT-4o and Llama 3.1 8B models**: In addition to GPT-3.5 Turbo, we built issue classifiers with the Llama 3.1 (8B parameters) and GPT-4o models. We compared the issue classification accuracy of these models. Our study is the first to apply the Llama 3.1 8B model to GitHub issue report classification.
- **Comparing the accuracy of project-specific and project-agnostic models**: We trained and evaluated a project-specific model and a project-agnostic model to quantitatively analyze the accuracy differences between the two models.

The remainder of this paper is organized as follows. Section II presents the background of this study. Section III discusses the related work. Section IV discusses how our paper differs from the original study, even if we conducted a replication study. Section V shows how we conducted our experiments. Section VI presents the results. Section VII discusses threats to validity. Section VIII discusses the results of this study. Section IX concludes.

II. Background

A. GitHub Issue Report Classification

This study focuses on classifying issue reports on GitHub. Figure 1 shows an example of GitHub's issue report. An issue report consists of a title, body, and labels. The title is a concise summary of the core content of the issue, and the body is a detailed description of the issue, showing the specific information needed to reproduce and resolve the issue. The labels indicate the characteristics of the issue and are used to organize the issue into categories for easy management.

GitHub projects manage their custom labels to represent the unique characteristics of each project. As a result, issue classifiers trained on project-specific issues tend to perform well. However, for labels that are commonly used across all projects, a project-agnostic model would be more suitable for issue classification.

We extend the OpenAI GPT-3.5 Turbo-based issue classification experiment conducted by Aracena et al. [14], which ranked 5th in the issue report classification competition of third Natural Language-Based Software Engineering (NLBSE) Workshop in 2024 [1]. Even though the model demonstrated relatively poor accuracy, the effectiveness and accuracy of GPT-based models have not yet been fully explored in the issue classification task. On the other hand, the BERT-based and Sentence Transformers-based models have already been extensively explored in issue classification research. In addition, BERT models specific to the software engineering and issue domain exist [9], [10]. The accuracy and effectiveness of the BERT-based issue classification approaches have already been well demonstrated.

In this paper, we further explore the classification accuracy of GPT-based models. We experiment with other GPT models such as Llama 3.1 8B and GPT-4o with additional datasets. We additionally evaluate the accuracy of project-specific classifiers as well as project-agnostic classifier.

B. Instruction Tuning

Instruction tuning is a learning method [12] that combines in-context learning and fine-tuning techniques. Wei et al. [12] showed that instruction tuning enables large language models to perform well on a variety of tasks. Wei et al. [12] reported that explicitly providing options (i.e., labels and classes) is effective in improving the accuracy of classification tasks. Instruction tuning enables a model to learn task objectives and data through structured instructions, enhancing its ability to generate appropriate

[1] https://nlbse2024.github.io/

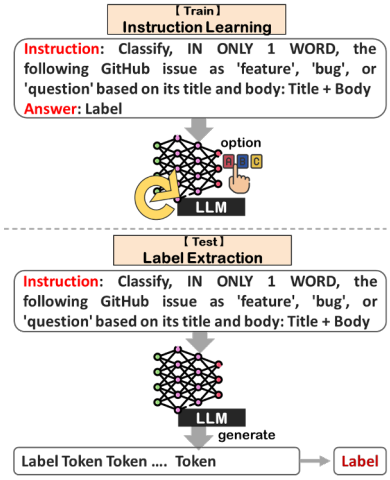

Fig. 2. Workflow of instruction tuning and label extraction for classifying GitHub issues into the categories of 'bug', 'feature', or 'question'

responses. By presenting tasks in a well-structured and interpretable manner, instruction tuning improves the model's adaptability across various tasks, even with limited data. An essential aspect of instruction tuning is the design of effective prompts, which integrate input data and task-specific commands to guide the model. Fine-tuned models tend to enhance accuracy in structured classification tasks, including issue classification. Our study fine-tunes the GPT model, following the method used in the previous study [14]. Figure 2 shows the training and testing process involved in instruction tuning. During training, we paired instructions and labels for issue classification tasks. Later, when using the trained model, we provided the same instructions and issue report data for classification.

III. Related Work

We focus on issue classification studies using Transformer models such as BERT and GPT.

A. BERT

Wang et al. [15] pointed out that traditional word embedding techniques do not account for polysemy issues and do not capture high-level information due to their single layer. Wang et al. measured and compared multi-label classification accuracy with four deep learning models, BERT, Bi-LSTM, CNN, and RCNN. Their experimental results showed that the F1-score@2 of BERT was the best with 37.4% when using a large amount of issue data, and the additional pre-trained model on StackOverflow outperformed the original BERT by 5.5% over the existing BERT.

Siddiq et al. [16] pointed out that manually tagging labels in GitHub's issue tracking system is time-consuming and error-prone. They proposed a BERT-based approach that classifies issues into questions, bugs, and enhancements. They fine-tuned their BERT model using data from 800,000 issue reports and achieved an average F1-score of 85.7%.

Colavito et al. [17] classified issue reports into bug, feature, and question. Colavito et al. proposed a two-stage classification structure. In the first stage, the issue title and body are used to obtain embedding values from RoBERTa. In the second stage, RoBERTa embeddings and issue author information (Author Association) are combined to build a multilayer perceptron (MLP) based classifier. The experimental result demonstrates an F1-score of 85.9%.

Similar to Colavito et al. [17], Bharadwaj et al. [18] designed a linear classifier by combining the embedding values from RoBERTa model and feature information from Hand-crafted features (early issue , owner, and issue title) and showed an F1-score of 86.5%.

Trautsch et al. [19] fine-tuned seBERT [9], a pre-trained model in the software engineering domain, to classify issue reports into three classes: bugs, enhancements, and questions. The experimental results show an average F1-score (micro) of 85.7%, which is 4.1% better than the original FastText.

Gomes et al. [20] proposed a classification technique for predicting long-lived bugs in Free/Libre Open Source Software (FLOSS) using BERT and TF-IDF as feature extractors. The experimental results showed that SVM and Random Forest performed best when using BERT as a feature extractor.

Hadi et al. [21] proposed a classification technique utilizing pre-trained language models for the automatic classification of mobile application user feedback. Their experimental results showed that the pre-trained models had high accuracy across all datasets in binary and multi-class classification, with ALBERT and RoBERTa performing particularly well.

Zhifang et al. [22] proposed PIFTNet, a GitHub bug report classification method that uses transfer learning to classify GitHub issue reports. The experimental results showed that PIFTNet outperformed FastText, ATTBiLSTM, TextLSTM, TextANN, TextBiLSTM, and TextCNN achieving an F1-score of 85.9%.

Heo et al. [10] pointed out that pre-trained models in general domain such as RoBERTa [6] in issue categorization have limitations in understanding the context of software engineering tasks. Heo et al. proposed IssueBERT and demonstrated that the binary classification of bug and feature had an average F1-score of 89.2%, and the multi-label classification had an average accuracy of 68.5%. This is 1.74% higher than seBERT [19] and 3.61% higher than RoBERTa [6].

BERT-based classifiers have excellent sentence understanding capabilities and perform best in classifying issue reports. However, creating issue classifiers using BERT-based models requires a large amount of issue data per project, and optimizing the accuracy of BERT models requires building project-specific classifiers,

which is time-consuming and data-intensive.

B. GPT

Aracena et al. [14] proposed a classification technique utilizing GPT-3.5 Turbo to classify GitHub issue reports into bugs, features, and questions. They fine-tuned the GPT-3.5 Turbo model using the title and body of the training data as part of the prompts, with the label as the correct answer. The experimental results showed that the precision was 83.24%, recall was 82.87%, and F-measure was 82.8%, indicating the effectiveness of the GPT-based model and the GitHub issue classification technique using prompts.

Colavito et al. [23] proposed an automated technique for issue report classification utilizing large-scale language models such as GPT. They utilized GPT-3.5 Turbo and compared the results of multi-class classification of issue reports under few-shot and zero-shot settings with SETFIT, a BERT-based model. The experimental results showed that GPT-3.5 Turbo performed 0.8155 in F1-micro and 0.8095 in F1-macro, while SETFIT performed slightly better with 0.8321 in F1-micro and 0.8246 in F1-macro. It is important to note that unlike the fine-tuned SETFIT model, GPT-3.5 Turbo achieved similarly in the zero-shot setting without fine-tuning, demonstrating its ability to classify issue reports even with unlabeled data.

Colavito et al. [24] also compared the SETFIT model's few-shot learning, RoBERTa, and GPT-3.5 zero-shot learning results. Those models achieved F1-scores of 83.2%, 81.8%, and 81.5%, respectively. The results show that GPT-based models can achieve similar accuracy to the state-of-the-art without fine-tuning.

Du et al. [25] proposed a bug report classification framework for deep learning frameworks (LLM-BRC). The framework utilizes OpenAI's latest embedding model to improve the accuracy of bug classification. The LLM-BRC framework classifies issue reports to bug or non-bug. Experimental results showed F1-scores of 92% to 98.75% for three deep learning frameworks: TensorFlow, MXNET, and PaddlePaddle.

Since GPT models are pre-trained with much larger datasets compared to BERT, they can perform well without additional fine-tuning or with only a small amount of data. This is especially efficient when the size of a project is small and there is limited issue data available. With GPT-based models, issue classifiers can yield good accuracy without project-specific models. However, GPT-based issue classification tasks have not been fully explored yet. Therefore, in this study, we replicate Aracena et al. study [14] to evaluate the accuracy of GPT-based issue classification tasks.

IV. Study Replication

In this study, we replicate a previous study [14] that utilized the API for issue classification in LLM. While the issue classification performed well using OpenAI's API, there is a need to evaluate accuracy on a wider range of models. Therefore, in this study, we extend the replication. Table I summarizes the research design of both experiments.

TABLE I
Comparison of a previous study [14] and ours

Category	Base	Ours
Project	5 Corpus [26]	5 Corpus [26] additional 6 project
Dataset (train:test)	1500:1500 (50%:50%)	3300:3300 (50%:50%)
Preprocessing	CM1, CM2	CM1
Model	GPT-3.5 Turbo	GPT-3.5 Turbo-0125, GPT-4o 2024-08-06, Llama 3.1 8B Instruct
Project-Integration model	X	O
Label	bug, feature, question	

As shown in Table I, Aracena et al.'s study [14] and this study used the GitHub issue dataset published by [26]. Aracena et al. study [14] performed issue classification tasks on five projects from the repository of nlbse2024[2]. In this replication study, we collected issue reports from six additional projects. To select the six projects, we sorted the open projects on GitHub by number of issue reports and selected the top 100 projects. We then manually investigated if those projects maintained the labels "Bug", "Feature", and "question" on issue reports from the first-ranked project to the lower-ranked project. The details are in our previous work [10]. We collected a total of 11 projects.

When it comes to GPT models, Aracena et al. study [14] used the GPT-3.5 Turbo model for issue classification. In addition, we further explore the issue classification accuracy of GPT-4o and the open-source Llama 3.1 8B model. Finally, we develop and evaluate the accuracy of the project-agnostic model to measure the accuracy of three commonly used labels (Bug, Feature, and Question) across GitHub projects. We present more details in Section V.

V. Experimental setting

A. Research Questions

This replication study has three main objectives. First, we evaluate the original GPT model with additional data. Second, we evaluate different GPT models. Third, we explore the accuracy of a project-agnostic model. Therefore, we set up the following research questions.

- RQ1. What is the accuracy of issue classification with OpenAI GPT-3.5 Turbo as well as GPT-4o added in this study?
- RQ2. What is the accuracy of issue classification with Llama 3.1 8B model?

[2]https://github.com/nlbse2024/issue-report-classification

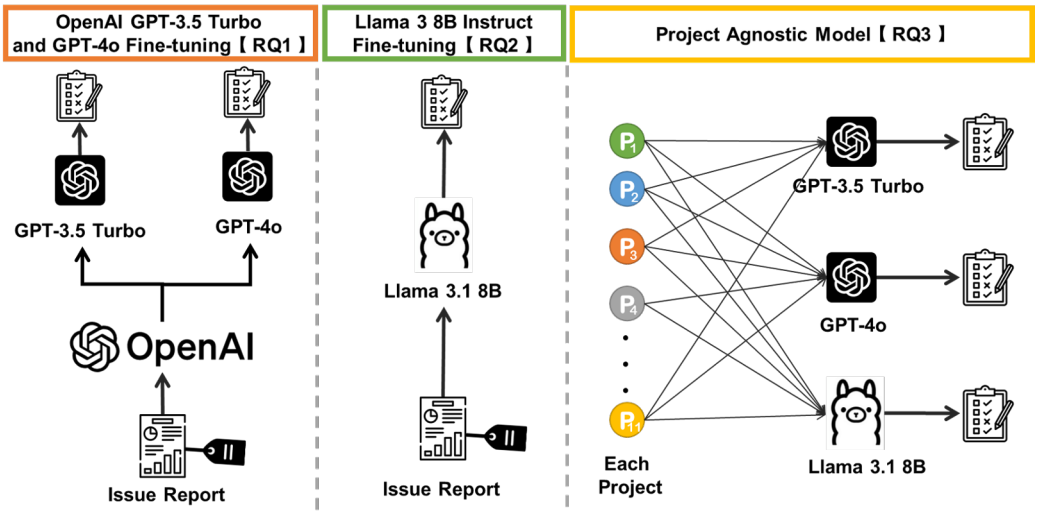

Fig. 3. An overview of constructing GPT-based issue classifiers for RQ1, RQ2, and RQ3.

- RQ3. What is the accuracy of issue classification for not only for the project-specific model but for the project-agnostic model?

RQ1 aims to evaluate the accuracy of a replication experiment using issue classifiers based on the GPT-3.5 Turbo model and the GPT-4o model. By doing so, we intend to analyze the accuracy differences between the two models and determine which is more suitable for issue classification tasks. This experiment focuses on determining if the improved accuracy of the latest LLM (GPT-4o) actually makes a meaningful difference in the issue classification task.

RQ2 aims to evaluate the feasibility of an issue classifier utilizing an open-source pre-trained model Llama 3.1 8B. This study analyzes the accuracy by training an issue classifier trained with the Llama 3.1 8B model[3] instead of commercial models (GPT series). By doing so, we verify whether it is possible to build a high-performing issue classifier even with a publicly available model.

RQ3 aims to compare the difference in accuracy between a project-specific issue classifier (Project-Specific Model) and a unified model that is generally applicable across multiple projects (Project-Agnostic Model). Due to the nature of issue management on GitHub, different projects may define and use labels differently, which requires models optimized for these variations. However, common labels such as 'Bug', 'Feature', and 'Question' have the potential to perform well when trained with a unified model. This study trains such a unified model and evaluates the practicality and accuracy of a project-agnostic, general-purpose classifier compared to project-specific models.

[3]We conducted the on a computer equipped with NVIDIA GeForce RTX 3090 x 4, Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz, 64GB RAM.

B. Github Issue Data

Aracena et al. study [14] used the issue report dataset provided by NLBSE'23 Tool Competition on Issue Report Classification[4]. It consists of train set, test set, and 3 labels for 5 projects as shown in Table II. In addition, we collected data from six more projects to evaluate the accuracy of issue report classification on more projects. The additional data of six projects we collected can be found at https://github.com/jueun4136/issue-data.git.

TABLE II
ISSUE DATA OF OUR STUDY

Category	Base [14]	Ours
project name	Facebook, Tensorflow, Microsoft, Bitcoin, Opencv	Ansible, Flutter, Kubernetes, Roslyn, Typescript, Dartlang
# of issue per project(train:test)	300:300 bug: 100, feature: 100, question: 100	
# of total issue(train:test)	3300:3300	

Because the dataset in this study has the same number of instances per class, there is no data imbalance.

C. Data Preprocessing

Issue reports contain special characters, templates, logs, HTML tags, etc. Also, it is needed to pre-process issue reports to avoid exceeding the length of the model input. Aracena et al. [14] used two different pre-processing methods as Clean Method 1 and Clean Method 2. The procedure of Clean Method 1 (CM1) applied by Alqahtani et al. [27] is as follows. Remove punctuation, certain template sentences, emoji, URL, HTML, special characters, spaces, line breaks, and

[4]https://github.com/nlbse2024/issue-report-classification

words larger than 20 characters, change case, and concatenate Title and Body. The procedure of Clean Method 2 (CM2) removes more string patterns, processes tags, and removes markdown syntax.

For our study, we selected CM1 to pre-process the issue report data, since the experiments in the Aracena et al. study [14] showed that three out of five projects had good classification performance on CM1.

D. GPT Models

The previous study [14] used GPT-3.5 Turbo-0125 to classify issues using LLM. In addition to GPT-3.5 Turbo-0125, we used GPT-4o-2024-08-06 and Llama-3.1-8B-Instruct[5] in our replication study. The specifications of each model are shown in Table III.

While GPT-3.5 Turbo excels in efficiency and fast response time, GPT-4o shows strengths in complex logical reasoning and multi-step task processing. Llama 3.1 8B, on the other hand, performs well on small datasets and specific domain tasks due to its lightweight architecture. These differences are used to quantitatively evaluate and analyze the accuracy and strengths of each model in the issue classification task.

We constructed each GPT-based model as shown in Figure 3. For RQ1, we fine-tuned OpenAI's GPT-3.5 Turbo and GPT-4o using the Instruction tuning technique. For RQ2, we fine-tune the Llama 3.1 8B model. RQ1 and RQ2 are focus on generating project-specific classifiers. For RQ3, we integrated all data from eleven projects to generate a project-agnostic classifier.

The hyperparameters used for model building and experimentation are as follows: GPT-3.5 Turbo and GPT-4o sets up n_epochs to 3 and batch_size to 1 for training and max_content_tokens to 3,999 for testing.

In the case of Llama 3.1 8B, we loaded the model from Hugging Face and performed PEFT with 4-bit quantization and LoRA settings. For training, we set num_train_epochs to 3 and per_device_train_batch_size to 1 as hyperparameters. For testing, we set temperature to 0.1 to ensure that the model produces consistent answers. Detailed configuration values and source code used in the experiment can be found in the following repository[6].

E. GPT-based models Instruction Tuning

To build a GPT-based model for classifying issue reports, we fine-tuned each model using the instruction tuning technique. We applied the same instruction as [14] to GPT-3.5 Turbo, GPT-4o, and Llama 3.1 8B Instruct. During training, we provided an instruction with classification options (labels), issue report information (Title+Body) and the correct answer (Label) as a prompt. In the testing phase, we provided the same instruction with a different issue report information to the trained model. We extracted the first word from the model's output and used it as the final prediction.

[5]https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

[6]https://github.com/jueun4136/issue-data.git

```
{
   messages: [
        {role: system, content: GitHub Issue
        ↪  Report Classifier},
        {role: user, content: Classify, IN ONLY 1
        ↪  WORD, the following GitHub issue as
        ↪  'feature', 'bug', or 'question' based
        ↪  on its title and body: Title + Body},
        {role: assistant, content: Label}
    ]
}
```

Fig. 4. Prompt Template of GPT-3.5 Turbo and GPT-4o

```
<|begin_of_text|><|start_header_id|> system
↪  <|end_header_id|> GitHub Issue Report
↪  Classifier <|eot_id|><|start_header_id|> user
↪  <|end_header_id|> Classify, IN ONLY 1 WORD,
↪  the following GitHub issue as 'feature',
↪  'bug', or 'question' based on its title and
↪  body: + Title, Body +
↪  <|eot_id|><|start_header_id|> assistant
↪  <|end_header_id|>+ Label
↪  +<|eot_id|><|end_of_text|>
```

Fig. 5. Prompt Template of Llama 3.1 8B

F. Prompt Generation

Since prompt design significantly impacts model accuracy, this section explains how to construct a prompt. For GPT models, it is important to configure prompts to match the format of the pre-trained model prompts. In other words, consistency in prompt format between the pre-trained model and the fine-tuned model is important. Since the format of prompts varies from model to model, we needed to generate different prompts for different models.

First, to implement issue classifiers with GPT-3.5 Turbo and GPT-4o models, we followed OpenAI's instructions[7]. We then configured the prompts as shown in Figure 4. In these prompts, we assigned the system the role of a GitHub issue classifier. We then provided the title and body of the issue and asked the system to classify the issue as either feature, bug, or question based on the given content.

Next, to implement an issue classifier with Llama 3.1 8B, we followed Meta's instructions[8] and configured it with the prompts shown in Figure 5.

Like GPT models, the prompts of Llama 3.1 8B use a conversational model that can manage conversations with a role-based structure (system, user, assistant). Figure 4 and Figure 5 define the input data for training in the conversational model and represent the entire training example, including both the 'prompt' and 'response'.

In addition, we conducted experiments using simple prompts to evaluate the impact of the prompt configuration in Llama 3.1 8B on classification accuracy. The

[7]https://cookbook.openai.com/examples/how_to_format_inputs_to_chatgpt_models?utm_source=chatgpt.com

[8]https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_1/

TABLE III
GPT-3.5 TURBO, GPT-4O, LLAMA 3.1 (8B INSTRUCT) MODELS

Model	GPT-3.5 Turbo-0125	GPT-4o-2024-08-06	Llama 3.1 (8B Instruct)
Parameters	No external disclosure	No external disclosure	8B
Context Window	16,385 tokens	128,000 tokens	128,000 tokens
Max tokens	4,096 tokens	16,384 tokens	2,048 tokens
Training data	general-purpose internet data	Expanded general-purpose internet and high-quality data	Community-driven code and text data
Instruction Tuning	Limited support	Optimization and scaling	Command-centric
Fine-tuning	Supported	Supported	Fully customizable with user-provided data
Price(Training)	$8 per 1M tokens	$25 per 1M tokens	Free (requires user-provided computational resources)
Price(In/Out)	$3 / $6 per 1M tokens	$3.75 / $15 per 1M tokens	

configuration of simple prompts is shown in Figure 5, where the model was trained and tested using system role and issue report information but excluding a user instruction.

G. Evaluation Metrics

We used the classification_report function from the sklearn.metrics library to evaluate the accuracy of our multi-class issue report classification task. This function provides detailed metrics such as Precision, Recall, F1-Score and, macro average for each class.

VI. EXPERIMENTAL RESULT

We discuss the results of our experiments in subsections organized by research question.

A. RQ1. What is the accuracy of issue classification with OpenAI GPT-3.5 Turbo as well as GPT-4o added in this study?

The experimental result for RQ1 presents the issue classification accuracy by replicating Aracena et al. study [14]. In addition, this experimental result shows the accuracy on additional data of six projects as well as the accuracy of the advanced GPT model GPT-4o on the data of eleven projects. Figure 6 shows average F1-scores of GPT models across eleven projects, where GPT-4o achieved higher F1-scores than those of GPT-3.5 Turbo across ten projects.

Table IV shows the experimental result in detail. We highlighted the highest F1-scores for each project in bold and underlined the second-highest scores. In the table, when we compared the results of GPT-3.5 Turbo in our replication study with Aracena et al. study [14], we found few differences of F1-scores: +0.0040 for Facebook, -0.0054 for Tensorflow, -0.0034 for Microsoft, +0.0243 for Bitcoin, and -0.0122 for Opencv. We concluded that our replication study yields similar accuracy to that of Aracena et al. study [14].

When we compared the results of our replication study with additional data, the GPT-3.5 Turbo model yielded 0.8869 for Kubernetes, 0.8891 for Flutter, 0.7826 for Roslyn, 0.9735 for Ansible, 0.7473 for Typescript and 0.8866 for Dartlang. The highest accuracy of the GPT-3.5 Turbo model was 0.9735 for Ansible and the lowest accuracy was 0.7473 for Typescript. Considering that the highest accuracy from the original data was 0.8654 for Tensorflow and the lowest was 0.7908 for Bitcoin, we could understand that the accuracy of Aracena et al. study [14] is valid, while the variations have increased.

The GPT-4o model outperformed the GPT-3.5 Turbo model in overall. The GPT-4o yielded an average F1-score of 0.8718 for Facebook, 0.8750 for Tensorflow, 0.8433 for Microsoft, 0.8006 for Bitcoin, and 0.8179 for Opencv. The highest accuracy was 0.8750 for Tensorflow, and the lowest accuracy was 0.8006 for Bitcoin. The highest and lowest performing projects were the same, but the F1-scores went up to 0.0096 and 0.0098, respectively.

The GPT-4o model outperformed the GPT-3.5 Turbo model on the additional data as well. The GPT-4o model yielded 0.9149 for Kubernetes, 0.8992 for Flutter, 0.8167 for Roslyn, 0.9867 for Ansible, 0.7967 for Typescript, and 0.8800 for Dartlang. The highest performing project was Ansible at 0.9867, while the lowest performing was Typescript at 0.7967. the F1-

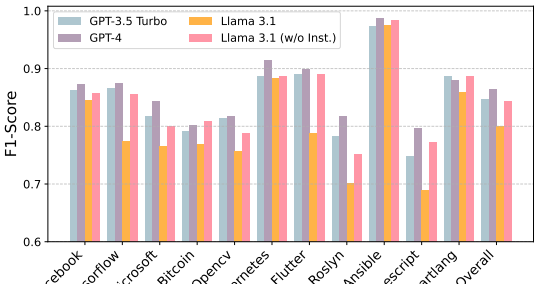

Fig. 6. F1-scores of GPT-3.5 Turbo, GPT-4o, Llama 3.1 8B, and Llama 3.1B (w/o Instruction) per each project

TABLE IV
RESULT OF ISSUE REPORT CLASSIFICAITON

		GPT 3.5-turbo			GPT-4o			Llama 3.1 8B			Llama 3.1 8B(w/o Inst.)		
Repo	label	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Facebook	bug	0.8846	0.9200	0.9020	0.8559	0.9500	0.9005	0.9020	0.9200	0.9109	0.8559	0.9500	0.9005
	feature	0.8273	0.9100	0.8667	0.8407	0.9500	0.8920	0.7797	0.9200	0.8440	0.8214	0.9200	0.8679
	question	0.8837	0.7600	0.8172	0.9600	0.7200	0.8229	0.8750	0.7000	0.7778	0.9221	0.7100	0.8023
Facebook	average	0.8652	0.8633	0.8619	0.8855	0.8733	**0.8718**	0.8522	0.8467	0.8442	0.8665	0.8600	0.8569
Tensorflow	bug	0.9239	0.8500	0.8854	0.8980	0.8800	0.8889	0.9222	0.8300	0.8737	0.9043	0.8500	0.8763
	feature	0.9643	0.8100	0.8804	0.9765	0.8300	0.8973	0.7200	0.9000	0.8000	0.9639	0.8000	0.8743
	question	0.7500	0.9300	0.8304	0.7778	0.9100	0.8387	0.7059	0.6000	0.6486	0.7398	0.9100	0.8161
Tensorflow	average	0.8794	0.8633	0.8654	0.8841	0.8733	**0.8750**	0.7827	0.7767	0.7741	0.8693	0.8533	0.8556
Microsoft	bug	0.8200	0.8200	0.8200	0.8614	0.8700	0.8657	0.7383	0.7900	0.7633	0.8352	0.7600	0.7958
	feature	0.7768	0.8700	0.8208	0.7982	0.8700	0.8325	0.7593	0.8200	0.7885	0.7615	0.8300	0.7943
	question	0.8636	0.7600	0.8085	0.8778	0.7900	0.8316	0.8118	0.6900	0.7459	0.8100	0.8100	0.8100
Microsoft	average	0.8201	0.8167	0.8164	0.8458	0.8433	**0.8433**	0.7698	0.7667	0.7659	0.8022	0.8000	0.8000
Bitcoin	bug	0.8352	0.7600	0.7958	0.8172	0.7600	0.7876	0.7778	0.7000	0.7368	0.7652	0.8800	0.8186
	feature	0.8416	0.8500	0.8458	0.8800	0.8800	0.8800	0.8302	0.8800	0.8544	0.8776	0.8600	0.8687
	question	0.7037	0.7600	0.7308	0.7103	0.7600	0.7343	0.7019	0.7300	0.7157	0.7931	0.6900	0.7380
Bitcoin	average	0.7935	0.7900	0.7908	0.8025	0.8000	0.8006	0.7700	0.7700	0.7690	0.8120	0.8100	**0.8084**
Opencv	bug	0.7822	0.7900	0.7861	0.8256	0.7100	0.7634	0.9216	0.4700	0.6225	0.9153	0.5400	0.6792
	feature	0.8224	0.8800	0.8502	0.8155	0.8400	0.8276	0.7692	0.9000	0.8295	0.8614	0.8700	0.8657
	question	0.8370	0.7700	0.8021	0.8198	0.9100	0.8626	0.7197	0.9500	0.8190	0.7000	0.9800	0.8167
Opencv	average	0.8139	0.8133	0.8128	0.8203	0.8200	**0.8179**	0.8035	0.7733	0.7570	0.8255	0.7967	0.7872
Kubernetes	bug	0.8817	0.8200	0.8497	0.8980	0.8800	0.8889	0.8889	0.8000	0.8421	0.7966	0.9400	0.8624
	feature	0.9588	0.9300	0.9442	0.9596	0.9500	0.9548	0.9681	0.9100	0.9381	0.9694	0.9500	0.9596
	question	0.8273	0.9100	0.8667	0.8922	0.9100	0.9010	0.8103	0.9400	0.8704	0.9167	0.7700	0.8370
Kubernetes	average	0.8893	0.8867	0.8869	0.9166	0.9133	**0.9149**	0.8891	0.8833	0.8835	0.8942	0.8867	0.8863
Flutter	bug	0.8673	0.8586	0.8629	0.9326	0.8300	0.8783	0.8750	0.5600	0.6829	0.8854	0.8500	0.8673
	feature	0.9320	0.9600	0.9458	0.8981	0.9700	0.9327	0.8348	0.9600	0.8930	0.9375	0.9000	0.9184
	question	0.8673	0.8500	0.8586	0.8738	0.9000	0.8867	0.7190	0.8700	0.7873	0.8519	0.9200	0.8846
Flutter	average	0.8889	0.8895	0.8891	0.9015	0.9000	**0.8992**	0.8096	0.7967	0.7878	0.8916	0.8900	0.8901
Roslyn	bug	0.7544	0.8600	0.8037	0.8302	0.8800	0.8544	0.6336	0.8300	0.7186	0.7282	0.7500	0.7389
	feature	0.8602	0.8000	0.8290	0.8710	0.8100	0.8394	0.7890	0.8600	0.8230	0.8723	0.8200	0.8454
	question	0.7419	0.6900	0.7150	0.7525	0.7600	0.7562	0.7500	0.4500	0.5625	0.6602	0.6800	0.6700
Roslyn	average	0.7855	0.7833	0.7826	0.8179	0.8167	**0.8167**	0.7242	0.7133	0.7014	0.7536	0.7500	0.7514
Ansible	bug	0.9259	1.0000	0.9615	0.9615	1.0000	0.9804	0.9709	1.0000	0.9852	0.9524	1.0000	0.9756
	feature	1.0000	0.9800	0.9899	1.0000	1.0000	1.0000	0.9434	1.0000	0.9709	1.0000	1.0000	1.0000
	question	1.0000	0.9400	0.9691	1.0000	0.9600	0.9796	0.9899	0.9800	0.9849	1.0000	0.9500	0.9744
Ansible	average	0.9753	0.9733	0.9735	0.9796	0.9867	**0.9867**	1.0000	0.9500	0.9744	0.9841	0.9833	0.9833
Typescript	bug	0.7526	0.7300	0.7411	0.7879	0.7800	0.7839	0.6848	0.6300	0.6562	0.7281	0.8300	0.7757
	feature	0.8021	0.7700	0.7857	0.7864	0.8100	0.7980	0.8298	0.7800	0.8041	0.8247	0.8000	0.8122
	question	0.6916	0.7400	0.7150	0.8163	0.8000	0.8081	0.5702	0.6500	0.6075	0.7753	0.6900	0.7302
Typescript	average	0.7488	0.7467	0.7473	0.7969	0.7967	**0.7967**	0.6949	0.6867	0.6893	0.7760	0.7733	0.7727
Dartlang	bug	0.8750	0.9100	0.8922	0.9053	0.8600	0.8821	0.8224	0.8800	0.8502	0.8776	0.8600	0.8687
	feature	0.9082	0.8900	0.8990	0.8835	0.9100	0.8966	0.8571	0.9600	0.9057	0.9118	0.9300	0.9208
	question	0.8776	0.8600	0.8687	0.8529	0.8700	0.8614	0.9136	0.7400	0.8177	0.8700	0.8700	0.8700
Dartlang	average	0.8869	0.8867	**0.8866**	0.8806	0.8800	0.8800	0.8644	0.8600	0.8579	0.8864	0.8867	0.8865
Overall	bug	0.8457	0.8471	0.8455	0.8703	0.8545	0.8613	0.8307	0.7645	0.7857	0.8404	0.8373	0.8326
	feature	0.8812	0.8773	0.8780	0.8827	0.8927	0.8864	0.8255	0.8991	0.8592	0.8910	0.8800	0.8843
	question	0.8222	0.8155	0.8165	0.8485	0.8445	0.8439	0.7788	0.7545	0.7579	0.8217	0.8164	0.8136
Overall	average	0.8497	0.8466	0.8467	0.8665	0.8639	**0.8639**	0.8146	0.8021	0.8004	0.8510	0.8445	0.8435

scores went up to 0.0101 and 0.0494, respectively.

On average, GPT-4o yielded an F1-score of 0.0172 higher than GPT-3.5 Turbo. GPT-4o yielded higher F1-scores than GPT-3.5 Turbo on 10 projects, ranging from +0.0051 to +0.0494. However, for one project, Dartlang, GPT-4o yielded lower F1-scores than GPT-3.5 Turbo (-0.0066).

Summary: The issue classification accuracy of GPT-3.5 Turbo was similar to that of the original study [14]. GPT-4o performed higher than GPT-3.5 Turbo for 10 projects and lower than GPT-3.5 Turbo for 1 project.

B. RQ2. What is the accuracy of issue classification with Llama 3.1 8B model?

The experimental result for RQ2 demonstrates the accuracy of the issue classifier generated by the Llama 3.1 8B Instruct model with and without an instruction. Figure 6 also shows average F1-scores of Llama models across eleven projects, where GPT-4o achieved higher F1-scores than those of Llama 3.1.

Table IV shows the experimental result in the right columns. When we compared the accuracy of Llama 3.1 8B with that of GPT-3.5 Turbo on the original data [26], Llama 3.1 8B yielded lower accuracy than that of GPT-3.5 Turbo across five projects. When it comes to the additional data of six projects, Llama 3.1 8B yielded lower accuracy than that of GPT-3.5 Turbo across five projects, while higher accuracy than that of GPT-3.5 Turbo for one project Ansible (+0.0009). The averaged F1-score of Llama 3.1 8B is 0.8004, while the averaged F1-score of GPT-3.5 Turbo is 0.8467.

Interestingly, the Llama 3.1 8B model without any instructions yielded a higher averaged F1-score of 0.8435 than that of the Llama 3.1 8B model with instructions (0.8004). The difference is +0.0431. This implies that longer prompts can cause accuracy degradation in a Llama model.

When we compared the accuracy of Llama 3.1 8B with that of GPT-4o, Llama 3.1 8B yielded lower accuracy than that of GPT-4o across all of the eleven projects. Llama 3.1 8B without any instructions yielded lower accuracy than that of GPT-4o Turbo across nine of the eleven projects. However, Llama 3.1 8B without any instructions yielded higher accuracy than that of GPT-4o Turbo across two of the eleven projects (Bitcoin and Dartlang).

After we observed that Llama 3.1 8B model without instructions yielded higher accuracy than Llama 3.1 8B model with instructions, we tried the experiments with and without instructions on GPT-3.5 Turbo and GPT-4o. We found that GPT-3.5 Turbo showed an average increase of 0.0173 in accuracy without instructions, while GPT-4o showed an average decrease of 0.0046 without instructions. Because, we found that the no instructions do not have a significant impact on the GPT models, we did not include detailed experimental results in the paper.

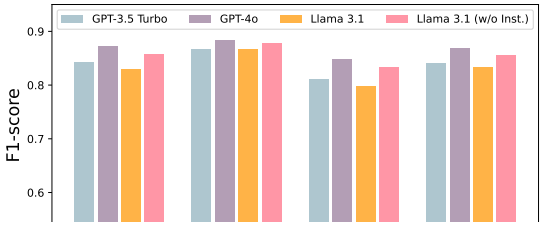

Fig. 7. Project-Agnostic F1-scores of GPT-3.5 Turbo, GPT-4o, Llama 3.1 8B, and Llama 3.1B (w/o Instruction) per each label

Summary: The issue classification accuracy of Llama 3.1 8B was consistently lower than that of GPT-4o across eleven projects. However, Llama 3.1 8B with no instructions yielded higher accuracy than that of GPT-4o Turbo across two of the eleven projects.

C. RQ3. What is the accuracy of issue classification for not only for the project-specific model but for the project-agnostic model?

The experimental result for RQ3 demonstrates the issue classification accuracy of project-agnostic models in general. Figure 7 shows the experimental result for RQ3, where GPT-4o outperforms other models.

Table V shows the experimental result in detail. We first compared an averaged F1-score of project-agnostic classifiers with that of project-specific classifiers. In the case of the GPT-3.5 Turbo model, the average F1-score of the project-agnostic classifier is 0.8400, while the project-specific classifier is 0.8467. The project-specific classifier yielded a higher average F1-score than the project-specific classifier.

In GPT-4o, the opposite result comes out. The average F1-score of the project-agnostic classifier is 0.8680, while that of the project-specific classifier is 0.8639. The average F1-score of the project-agnostic classifier is slightly higher than, or equal to that of the project-specific classifier.

Similar to GPT-4o, the project-agnostic classifier with Llama 3.1 8B is higher than that of the project-specific classifier. The average F1-score of the project-agnostic classifier is 0.8319, while that of the project-specific classifier is 0.8004.

More interesting are the results for Llama 3.1 8B (w/o Instruction) shown in Table V. The Llama 3.1 8B model outperformed the GPT-3.5 Turbo model (0.8558 vs. 0.8400).

Summary: The project-agnostic classifier with Llama 3.1 8B yields higher F1-scores than the project-specific classifier. The project-agnostic classifier withGPT-4o yields slightly higher F1-scores than the project-specific classifier.

TABLE V
RESULT OF 11 PROJECT AGNOSTIC MODEL

	GPT-3.5 Turbo			GPT-4o			Llama 3.1 8B			Llama 3.1 8B(w/o Instruction)		
label	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
bug	0.8212	0.8653	0.8427	0.8862	0.8572	**0.8715**	0.8115	0.8491	0.8299	0.8736	0.8418	0.8574
feature	0.8522	0.8809	0.8663	0.8621	0.9067	**0.8838**	0.8462	0.8900	0.8675	0.8904	0.8645	0.8773
question	0.8504	0.7754	0.8112	0.8564	0.8409	**0.8486**	0.8417	0.7591	0.7983	0.8072	0.8600	0.8327
average	0.8413	0.8405	0.8400	0.8683	0.8683	**0.8680**	0.8331	0.8327	0.8319	0.8571	0.8555	0.8558

VII. THREATS TO VALIDITY

Open source software repositories: We used eleven specific open source software (OSS) repositories as data sources for this study. However, it is possible that the repositories used in this study are not representative of all software repositories, as the content and structure of issue reports may vary depending on the characteristics of each project.

Prompt optimization: We used the prompt that used in Aracena et al. study [14]. However, the construction of the prompts can have a significant impact on the accuracy of issue classification tasks with GPT models. Since we have not experimented with prompts, there are possibilities that an optimal prompt would achieve higher accuracy.

hyperparameter tuning: In this study, we minimized the hyperparameter tuning of the Llama 3.1 8B model. However, there are various hyperparameters such as learning rate, batch size, number of epochs, and model size that can affect the accuracy. This could be one of the reasons that the Llama 3.1 8B model underperformed compared to the GPT models.

Model Transparency and Accessibility: Llama 3.1 8B is an open-source model with transparent training data and architecture, whereas GPT-3.5 Turbo and GPT-4o are closed-source models with undisclosed training data and processes, making accuracy analysis and comparison challenging. These differences can influence the interpretation of results, and the observed accuracy gaps may reflect disparities in model design and access to training data.

VIII. DISCUSSION

Our experiment results show that GPT-4o performed the best, followed by GPT-3.5 Turbo, and Llama 3.1 8B. There are a number of possible reasons for the lower accuracy of the Llama 3.1 8B model than the GPT models serviced by OpenAI. The first reason could be that the Llama 3.1 8B model used in this study has a significantly smaller number of parameters, compared to GPT-3.5 Turbo and GPT-4o. The second reasons could be that hyper-parameter tuning is not performed, resulting in suboptimal tuning results. The last reason could be that, while GPT models are tuned through Reinforcement Learning with Human Feedback (RLHF), the Llama 3.1 8B model are not tuned through RLHF. Therefore, the Llama 3.1 8B model without instructions may perform better than the Llama 3.1 8B model with instructions.

Even though the accuracy of the Llama 3.1 8B model was worse than those of GPT-3.5 Turbo and GPT-4o, a study on a Llama-based issue classifier is worth continuing. The Llama 3.1 8B model maintains a relatively small number of parameters and limited resources. However, it still performs competitively. In particular, the Llama 3.1 8B and Llama 3.1 8B(w/o Instruction) model performed better on one project (Ansible) and on four projects (Bitcoin, Flutter, Ansible, Dartlang), despite having a smaller architecture and parameters than GPT-3.5 Turbo. This shows the potential of the Llama 3.1 8B model as an efficient model to solve real-world problems. Llama 3.1 8B(w/o Instruction) model performed better than even GPT-4o on two project (Bitcoin and Dartlang).

Besides, the training time for each project-specific model took about 25-30 minutes on GPT-3.5 Turbo and GPT-4o. The training time for each project-specific model took about 40-45 minutes on Llama 3.1 8B. The training time for a project-agnostic model took 275-280 minutes on GPT-3.5 and GPT-4o, and 470 minutes on Llama 3.1 8B.

IX. CONCLUSION

In this study, we conducted a replication study of the previous research Aracena et al. [14]. We also carried out experiments with additional data and additional models. The experimental results show that GPT-4o demonstrates the best issue classification accuracy with an average F1-score of 0.8639, when the issue classifier is fine-tuned with project-specific data. In addition, GPT-4o achieves an average F1-score of 0.8680, when the issue classifier is fine-tuned with all of the issue data. Meanwhile, Llama 3.1 8B demonstrates the lower issue classification accuracy than those of GPT-3.5 Turbo and GPT-4o.

Future research could include optimizing prompts and applying N-shot techniques to improve classification accuracy across diverse project datasets. We also plan to fine-tune Llama 3.1's large-scale models with domain-specific datasets to further enhance the accuracy of issue classifiers, and extend this to issue classification agent research. In addition, we will extend the data labels to multi-labels or various custom labels to study issue classifiers that can be used in real-world practices.

REFERENCES

[1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[2] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.

[3] Q. Fan, Y. Yu, G. Yin, T. Wang, and H. Wang, "Where is the road for issue reports classification based on text mining?," in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 121–130, IEEE, 2017.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.

[5] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[6] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[7] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[8] J. Tabassum, M. Maddela, W. Xu, and A. Ritter, "Code and named entity recognition in stackoverflow," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4913–4926, 2020.

[9] J. Von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Transactions on Software Engineering*, 2022.

[10] J. Heo, G. Kwon, C. Kwak, and S. Lee, "A comparison of pretrained models for classifying issue reports," *IEEE Access*, 2024.

[11] Z. Shaheen, G. Wohlgenannt, and E. Filtz, "Large scale legal text classification using transformer models," *arXiv preprint arXiv:2010.12871*, 2020.

[12] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *arXiv preprint arXiv:2109.01652*, 2021.

[13] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *Advances in neural information processing systems*, vol. 30, 2017.

[14] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models api to issue classification problem," *arXiv preprint arXiv:2401.04637*, 2024.

[15] J. Wang, X. Zhang, and L. Chen, "How well do pre-trained contextual language representations recommend labels for github issues?," *Knowledge-Based Systems*, vol. 232, p. 107476, 2021.

[16] M. L. Siddiq and J. C. Santos, "Bert-based github issue report classification," in *2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pp. 33–36, IEEE, 2022.

[17] G. Colavito, F. Lanubile, and N. Novielli, "Issue report classification using pre-trained language models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pp. 29–32, 2022.

[18] S. Bharadwaj and T. Kadam, "Github issue classification using bert-style models," in *2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pp. 40–43, IEEE, 2022.

[19] A. Trautsch and S. Herbold, "Predicting issue types with sebert," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pp. 37–39, 2022.

[20] L. Gomes, R. da Silva Torres, and M. L. Côrtes, "Bert-and tf-idf-based feature extraction for long-lived bug prediction in floss: a comparative study," *Information and Software Technology*, vol. 160, p. 107217, 2023.

[21] M. A. Hadi and F. H. Fard, "Evaluating pre-trained models for user feedback analysis in software engineering: A study on classification of app-reviews," *Empirical Software Engineering*, vol. 28, no. 4, p. 88, 2023.

[22] L. Zhifang, W. Kun, Z. Qi, L. Shengzong, Z. Yan, and H. Jianbiao, "Classification of open source software bug report based on transfer learning," *Expert Systems*, vol. 41, no. 5, p. e13184, 2024.

[23] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Leveraging gpt-like llms to automate issue labeling," in *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, pp. 469–480, IEEE, 2024.

[24] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Large language models for issue report classification," 2024.

[25] X. Du, Z. Liu, C. Li, X. Ma, Y. Li, and X. Wang, "Llm-brc: A large language model-based bug report classification framework," *Software Quality Journal*, pp. 1–21, 2024.

[26] R. Kallis, G. Colavito, A. Al-Kaswan, L. Pascarella, O. Chaparro, and P. Rani, "The nlbse'24 tool competition," in *Proceedings of The 3rd International Workshop on Natural Language-based Software Engineering (NLBSE'24)*, 2024.

[27] S. S. Alqahtani, "Improve classification of security bug reports using fasttext. a machine learning based approach," 2022.